Arbres et forêts

Prévoir, suite — Séance 10

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

La semaine dernière, nous avons prévu avec les moindres carrés. Aujourd'hui nous changeons **l'estimateur, et rien d'autre** : mêmes données, mêmes variables, même découpage, mêmes repères.

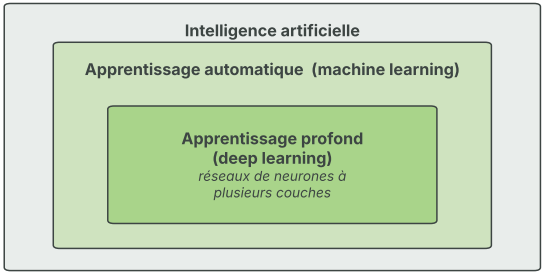
1. D'abord, regarder de près **les données** et les onze variables.
2. L'**arbre de décision** : une suite de questions, une constante par région.
3. La **forêt aléatoire** : des centaines d'arbres, et une moyenne.
4. La comparaison, et ce qu'on gagne — ou pas — à prendre un modèle plus souple.

La question de la séance

Un modèle plus puissant trouve-t-il ce que la régression n'a pas trouvé ?

Qu’est-ce que l’apprentissage automatique ?

Où tout cela se situe



Apprendre de l'expérience plutôt que suivre des règles écrites à l'avance. On ne programme pas la réponse : on fournit des exemples, l'algorithme en tire une règle.

Trois familles :

Supervisé — les exemples portent la *bonne réponse*. C'est tout notre cours.

Non supervisé — pas de réponse : on cherche des groupes ou des axes (ACP, classification automatique).

Par renforcement — on apprend par essais et récompenses.

Ce que vous faites depuis la séance 7

Une régression est déjà de l'apprentissage supervisé. Ce qui change aujourd'hui n'est pas la nature de l'exercice, c'est la souplesse du modèle.

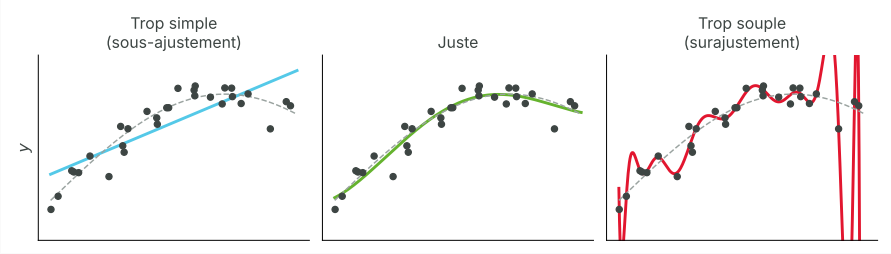
Les mêmes objets, deux vocabulaires

	En statistique	**En apprentissage automatique**
Ce qu'on explique	variable expliquée, endogène	**cible** (*target*, `y`)
Ce qui explique	variables explicatives, régresseurs	**variables** (*features*, `X`)
Une observation	un individu, une ligne	un *exemple*
Estimer	estimer le modèle	**entraîner** (*fit*)
Le résultat	des coefficients	un *modèle entraîné*
L'ajustement	R^2, résidus	**erreur** sur un jeu de test

- Les mots changent, les objets non. `X` et `y` sont les notations de `scikit-learn` — vous les retrouverez partout.
- Une différence réelle, toutefois : la statistique veut un **coefficient juste**, l'apprentissage veut une **erreur faible**. Ce n'est pas le même objectif, et cela change ce qu'on regarde.

La méthode, et les deux façons de se tromper

Quatre étapes, toujours les mêmes : découper les données → entraîner sur la partie apprentissage → prédire sur la partie test → mesurer l'erreur *là*, et nulle part ailleurs.



À gauche, le modèle est **trop simple** : il rate la forme même sur les données d'apprentissage — erreur de **biais**. À droite, il est **trop souple** : il passe par chaque point, donc il apprend le bruit — erreur de **variance**. Les deux prédisent mal, pour des raisons opposées, et c'est l'arbitrage central de la séance.

Les données

D’où viennent les données

```
brut = yf.download(["RNO.PA", "^FCHI"], start="2019-01-01",
                   end="2025-12-31", auto_adjust=True)
px = brut["Close"]
```

Date	MARCHE	TITRE
2019-01-02	99.41	100.39
2019-01-03	99.62	102.70
2019-01-04	98.46	98.55

- **Une ligne = une séance de bourse.** Environ 252 par an : ni les week-ends, ni les jours fériés. L’index n’est pas un calendrier continu.
- Deux colonnes seulement : le cours de clôture du **titre** et celui de l’**indice**. Tout le reste sera *construit*.

De deux colonnes à onze variables

```
r = np.log(px).diff()                              # les rendements
X["r_lag1"]  = r["TITRE"].shift(1)                 # hier
X["vol21"]   = r["TITRE"].shift(1).rolling(21).std()
y = r["TITRE"].abs()                               # la cible : l'amplitude
```

Date	r_lag1	r_lag2	ma5	vol21	marche1	lvolume
2019-02-01	-0.0051	0.0004	0.0058	0.0170	0.0019	14.93
2019-02-04	-0.0249	-0.0051	-0.0071	0.0176	-0.0120	16.64
2019-02-05	0.0216	-0.0249	-0.0027	0.0150	0.0203	15.31

Chaque variable porte un `.shift(1)` : elle doit être connue **la veille au soir**. Les 21 premières lignes disparaissent — une moyenne sur 21 jours a besoin de 21 jours.

Les onze variables, et pourquoi elles

Nom	Nombre	Ce qu'elle capte
`r_lag1` … `r_lag5`	5	les cinq derniers rendements : une éventuelle tendance courte
`ma5`, `ma21`	2	la direction moyenne récente, lissée
`vol5`, `vol21`	2	l'agitation récente — le regroupement de volatilité
`marche1`	1	ce qu'a fait le CAC 40 la veille
`lvolume`	1	l'intensité des échanges, en logarithme

Deux cibles, le même tableau de variables

`y = r` — le **rendement** de demain, son signe compris. `y = |r|` — son **amplitude** seule. La semaine dernière, la régression a échoué sur la première et réussi sur la seconde.

Le protocole, inchangé

```
c = int(len(d) * 0.8)
apprentissage, test = d.iloc[:c], d.iloc[c:]
```

	période	séances
apprentissage	fév. 2019 — mai 2024	1 382
test	mai 2024 — sept. 2025	346

- Découpage **chronologique**. Le repère reste la moyenne de l'apprentissage.
- Un R^2 hors échantillon **négatif** reste possible, et veut toujours dire la même chose : pire que de ne rien faire.

C'est tout l'intérêt de ne rien changer

Si les résultats diffèrent aujourd'hui, ce ne peut être que le modèle. Aucune autre explication n'est disponible.

L’arbre de décision

Dix jours, et une seule prévision

Dix journées tirées du fichier — deux dans chaque cinquième de volatilité, pour couvrir toute la plage. Deux colonnes seulement : ce qu'on sait le matin, ce qui arrive le lendemain.

Volatilité (%)	1,3	1,4	1,4	1,4	1,9	1,9	2,1	2,2	2,9	3,2
Amplitude (%)	0,7	0,5	0,2	0,1	2,8	2,8	0,9	2,4	2,7	4,0

Une seule prévision pour les dix jours : laquelle ?

La **moyenne**, $\bar{y} = 1{,}71\ \%$ — c'est la constante qui minimise la somme des carrés des écarts. Cette somme vaut 17,09. C'est notre point de départ : **l'erreur à battre**.

L’arbre ne fait rien d’autre que ceci

Couper l’axe en deux, prédire la moyenne de chaque côté, et mesurer ce qui reste. Il essaie **tous** les seuils possibles — il y en a six ici — et garde le meilleur.

Seuil	**À gauche**		**À droite**		**Erreur**	
	n	moyenne	*n*	moyenne	restante	
1,35	1	0,70	9	1,82	15,96	
1,65	**4**	**0,38**	**6**	**2,60**	**5,21**	← retenu
2,00	6	1,18	4	2,50	12,93	
2,15	7	1,14	3	3,03	9,58	
2,55	8	1,30	2	3,35	10,36	
3,05	9	1,46	1	4,00	11,26	

Le seuil **1,65** fait tomber l’erreur de 17,09 à 5,21 — il en retire 70 %. Aucun autre n’en approche. **C’est toute la mécanique de l’arbre**, répétée ensuite dans chaque morceau.

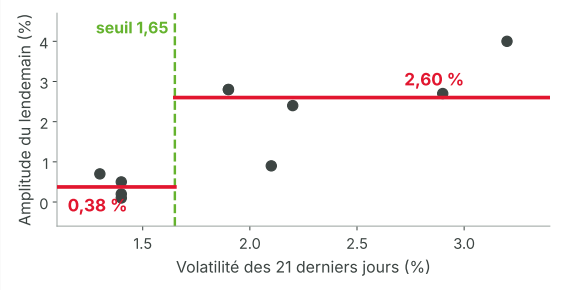
Le même calcul, en trois lignes

```
from sklearn.tree import
    DecisionTreeRegressor

X = vol.reshape(-1, 1)
a = DecisionTreeRegressor(
    max_depth=1)
a.fit(X, ampl)

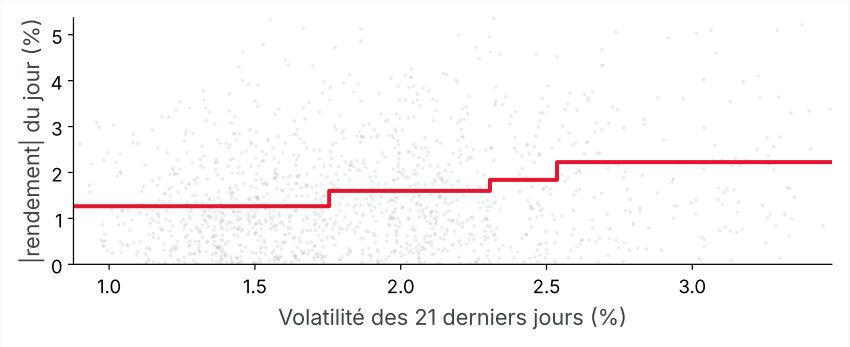
a.predict([[1.2], [2.5]])
# array([0.38, 2.60])
```

`max_depth=1` : **une seule** coupure. Ce sont les chiffres du tableau précédent, trouvés par la machine.



Avec `max_depth=2`, il recommencerait *dans chaque moitié* — à gauche à 1,35, à droite à 3,05. Quatre feuilles, quatre moyennes. Rien de plus.

L’idée : une constante par région



Le même calcul, sur les 1 382 jours. Trois coupures au lieu d’une, et la ligne rouge est le seul objet que l’arbre produit : une **moyenne par morceau**. Pas de droite, pas de coefficient — un escalier. Il ne suppose ni linéarité, ni additivité, ni continuité : on dit qu’il est **non paramétrique**. Il peut donc épouser n’importe quelle forme — y compris le bruit.

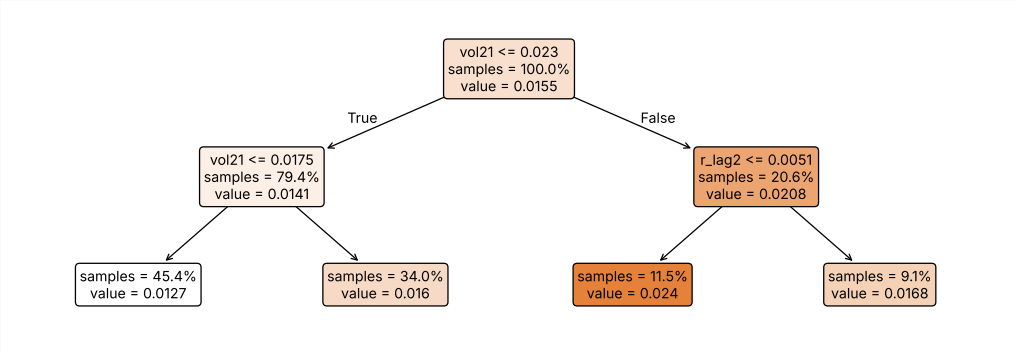
Le vocabulaire, sur l'exemple des dix jours

Le mot	Sur nos dix jours	Ce que c'est
Racine	« `vol21 <= 1,65` ? »	la première question
Nœud interne	les questions suivantes	chacune redécoupe un morceau
Feuille	0,38 % et 2,60 %	la prévision : une moyenne
Profondeur	1 ici	le nombre de questions enchaînées

Comment il choisit. Il essaie *toutes* les coupures — chaque variable, chaque seuil — et garde celle qui laisse le moins d'erreur, comme dans le tableau précédent. En classification, on remplace l'erreur par l'**indice de Gini**, qui mesure la même chose : l'homogénéité des groupes obtenus.

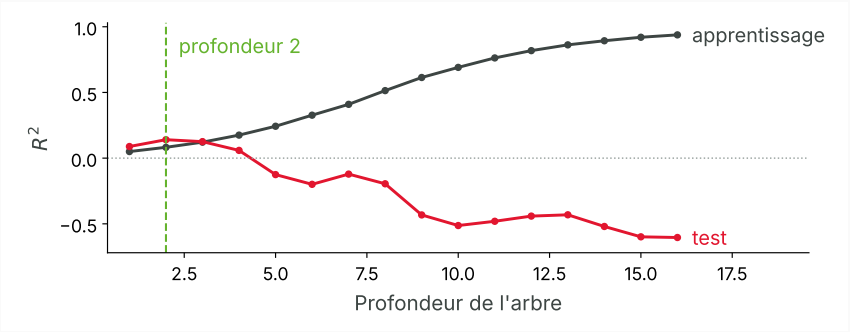
Quand s'arrête-t-il ? Jamais tout seul. Laissé libre, il coupe jusqu'à une feuille par observation — erreur nulle, et prévision inutile. C'est nous qui l'arrêtons : `max_depth`, ou une taille minimale de feuille.

L’arbre, en vrai



Profondeur 2, sur les onze variables. Il a choisi **tout seul** de commencer par la volatilité : `vol21 <= 0,023`. Si l’agitation récente est faible, il prédit **1,27 %** d’amplitude ; forte, **2,4 %**. Il a retrouvé le regroupement de volatilité sans qu’on le lui dise.

Plus profond n’est pas mieux



À la profondeur 16, l’arbre explique **94 %** de ce qu’on lui a montré — il a presque une feuille par journée. Sur des données nouvelles, il tombe à $-0{,}60$. Le meilleur arbre ici a **deux** niveaux : trois questions en tout.

La forêt aléatoire

Des centaines d'arbres, et une moyenne

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators=300, min_samples_leaf=20,
                           random_state=0).fit(X_app, y_app)
```

- Un arbre seul est **instable** : retirez vingt journées, il se réorganise entièrement.
- Une forêt en fait pousser des centaines — chacun sur un **tirage au sort** des observations, et à chaque question un **tirage au sort** des variables — puis elle **moyenne** leurs prédictions. C'est la seule idée.

	R^2 **hors échantillon**
20 arbres de profondeur 8, pris *un par un*	de $-0{,}63$ à $-0{,}06$
leur moyenne	$-0{,}29$
la forêt faite des mêmes arbres	$+0{,}15$

Le code, en entier

```
from sklearn.tree     import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

arbre = DecisionTreeRegressor(max_depth=3)
arbre.fit(X_app, y_app)                    # on entraine

foret = RandomForestRegressor(n_estimators=300, min_samples_leaf=20)
foret.fit(X_app, y_app)

print(arbre.score(X_test, y_test))         # R2 sur le test
print(foret.score(X_test, y_test))
```

- **Toujours les trois mêmes lignes** : créer le modèle, `.fit()`, puis `.score()` ou `.predict()`. Changer d'algorithme, c'est changer un seul mot.
- `.fit()` ne voit jamais le test. C'est toute la discipline, et elle tient dans cette phrase.
- Pour une **classification**, remplacez `Regressor` par `Classifier` : le reste est identique.

La comparaison

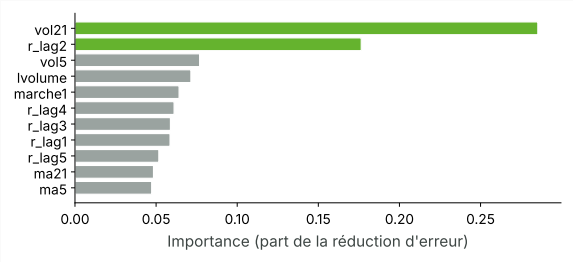
Trois modèles, deux cibles

Cible	Modèle	R^2 apprentissage	R^2 hors échantillon
rendement	régression	0,016	0,006
	arbre (prof. 3)	0,051	−0,169
	forêt	0,221	−0,022
amplitude $\lvert r \rvert$	régression	0,077	**0,156**
	arbre (prof. 3)	0,122	0,126
	forêt	0,278	0,146

- Sur le **rendement**, la forêt apprend quatorze fois mieux que la régression — et prévoit *moins bien que rien*. Quand il n'y a pas de signal, un modèle plus souple ne trouve pas davantage : il surajuste plus vite.
- Sur l'**amplitude**, où il y a du signal, les trois modèles se tiennent. La non-linéarité n'apporte rien ici, et c'est un résultat, pas un échec.

Chiffres du notebook du cours ; ils se rafraîchissent à chaque exécution.

Les importances ne sont pas des coefficients



Une **importance** dit de combien une variable a réduit l'erreur dans la forêt.

Elle n'a **ni signe, ni unité, ni écart-type**. Impossible d'écrire « $+1$ point de volatilité $\Rightarrow$ $+x$ ».

Elle est **biaisée** vers les variables à nombreuses valeurs distinctes, et se **partage** entre variables corrélées — deux variables jumelles paraissent toutes deux secondaires.

Ce qu'on perd en passant à la forêt

Pas d'intervalle de confiance, pas de test, pas d'effet marginal, aucune lecture causale. Une forêt prévoit bien et **n'explique rien** — elle ne répond pas à la question des séances 7 et 8.

Pour finir

Choisir, et ce qu'on retient

Une régression

- « quel est l'effet de . . . ? »
- un écart-type, un test, un intervalle
- un résultat qui se lit et se conteste

Une forêt

- « quelle sera la valeur ? »
- beaucoup de variables, formes inconnues
- personne n'a à interpréter le mécanisme

1. Ce n'est pas la puissance du modèle qui décide : c'est l'existence d'un **signal**.
2. Le **protocole** ne change pas d'un modèle à l'autre — c'est lui qui rend une comparaison possible.
3. Un **échec de prévision bien mesuré** est un résultat, pas une erreur d'étudiant.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?